LockCoop ：面向竞争锁交接前后的调度机制

摘要 在多核处理器上，Linux 调度器向可运行任务分配 CPU 时间，并选择任务的执行位置。虽然调度器会考虑 CPU 拓扑，但通常不会利用每把锁的等待关系和执行历史。当线程同时争用锁和 CPU 时间时，延迟持锁者的执行，也会延迟等待其所持锁的任务。任务被唤醒后，将其放到远端 CPU 上执行，可能增加访问锁及其保护数据的开销。例如，在多线程数据库服务器中，这些影响可能降低事务吞吐量并增加尾延迟。我们提出 LockCoop ，一个无需修改服务器软件、即可观测运行时锁事件的 Linux `sched_ext` 调度器。对于每把已观测到的锁，它记录等待任务；如果相应信息可用，还记录当前持锁者以及持锁者上一次运行的 CPU。它保留这些记录，供后续调度决策使用，并限制所存储的状态总量。它的两个调度机制 LockBoost 和 LockNest 使用相同的记录。解锁前，LockBoost 可以为经过验证且正在运行的持锁者提供有界的时间片扩展。唤醒后，LockNest 利用同一把锁的 CPU 历史，指导等待任务的 CPU 放置。如果某个动作所需的信息缺失或已经过期，则跳过该动作。我们使用互斥锁微基准和应用基准评估 LockCoop 。在一台具有 64 个物理核心、128 个逻辑 CPU 的 x86 服务器上，运行 256 个线程的 MariaDB OLTP 时，LockCoop 的吞吐量达到 `scx_simple` 基线的 4.14 倍，p95 延迟降低 82.2%。这些吞吐量和延迟方面的改善也出现在 ARM64、RISC-V 服务器及 VMware 虚拟机上。

关键词 操作系统；CPU 调度；锁竞争；锁交接；NUMA；缓存一致性；`sched_ext` ；锁因果调度

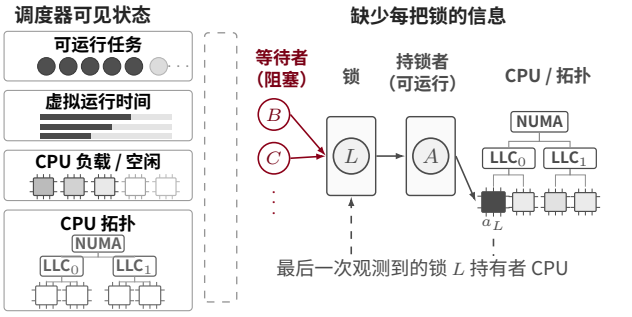


图 1 普通调度器能够观察任务和拓扑，却缺少及时调度持锁者和放置被唤醒等待者所需的信息。

1 引言

多线程服务器使用工作线程处理并发请求，并通过锁保护共享数据。当线程竞争 CPU 时间时，持锁者可能在释放锁之前被切换出去。这不仅会延误持锁者，也会延误其他等待同一把锁的请求。锁释放后，将被唤醒的等待任务放到远端 CPU 上执行，可能增加访问该锁及其保护数据的开销 [1]。因此，CPU 调度既影响请求等待多久，也影响同步操作消耗多少 CPU 时间。在数据库服务器中，这些开销可能降低事务吞吐量并增加尾延迟。

Linux 默认调度器会考虑可运行任务和 CPU 拓扑，但通常不追踪哪些任务等待同一把锁、哪个任务持有该锁，以及该锁的持有者最近在哪个 CPU 上运行（图 1）。因此，它可能在未考虑其他任务正在等待该锁的情况下切换持锁者，也可能在未利用同一把锁执行历史的情况下，为被唤醒的等待任务选择 CPU。

先前工作在不同决策位置处理上述开销。在锁实现内部，队列锁、Cohort、shuffling、CNA 与并发限制控制后继顺序或准入 [2-5, 1]；在调度器中，LCAS、CFL 与通用缓存感知策略作用于聚合竞争或任务级复用，无法区分每把锁的执行历史 [6-8]；通过显式接口，SCL 与 SynCord 传递同步语义 [9-10]。这些方法仍无法让操作系统调度器保留一把锁与其等待者、持有者和执行位置之间的关系。这些信息不能从持锁者服务一直保留到唤醒后的任务放置。因此，一个仍未解决的问题是：调度器能否从已有竞争路径恢复这些信息，并在不修改服务器软件的前提下作用于两个调度机会。

本文提出 LockCoop ，一个 Linux `sched_ext` 调度器。它从运行时锁事件中重建这些信息，将其保存在有界的调度器状态中，并在解锁前后使用这些状态。我们把这种方法称为锁因果调度。LockCoop 使用一条共享路径观测锁事件并保存状态，两个机制根据这些状态采取动作。LockCoop 记录等待每把观测到的锁的任务。在能够验证持锁者身份时，它还记录当前持锁者。此外，它记录最近一次观测到持锁者运行的 CPU。这些记录用于后续调度决策。持锁者字段指出哪个可运行任务能够释放被阻塞的等待者。CPU/拓扑字段记录同一把锁的持有者上一次在哪个 CPU 上运行。LockBoost 在解锁前采取动作，有限度地延长经过验证且受到延迟的持锁者的时间片。LockNest 在任务唤醒后采取动作，利用记录的 CPU 历史将被唤醒的等待者放置到该位置附近；被选中的 CPU 称为锚点。稳定的资源域将锚点限制在与工作负载相关的 CPU 中。完整的 LockCoop 调度器同时启用两个机制。为了测量各自的效果，我们也分别启用每个机制。

对于每个观测到的事件，LockCoop 记录现有证据能够支持持锁者时间片延长、后继放置或二者。记录状态会在陈旧后失效，并且状态规模与额外 CPU 服务均受到限制。没有可用证据时，LockCoop 沿用普通虚拟时间调度。

受控交接分别隔离两个调度动作，端到端实验则展示后继放置何时占主导，以及持锁者时间片延长何时提供额外收益。在一台具有 64 个物理核心、128 个逻辑 CPU 的 x86 服务器上，运行 256 个线程的 MariaDB OLTP 时，LockCoop 的吞吐量达到 `scx_simple` 的 4.14 倍，并将 p95 延迟降低 82.2%。在纯 OLTP 中，后继放置贡献了几乎全部收益；在 x86 CPU 干扰下，持锁者时间片延长进一步将 OLTP 吞吐量提高 9.6%–36.1%，同时不减少批处理工作量。

本文做出三点贡献：

1. 我们提出锁因果调度，为每把观测到的锁保留其等待者、持有者和执行位置之间的关系，并将其保存为有界的调度器状态。调度器在解锁前使用该状态推进持锁者，在唤醒后使用该状态放置被唤醒的等待者。

2. 我们在不修改服务器软件的前提下，在 Linux `sched_ext` 中设计并实现 LockBoost 与 LockNest ，即 LockCoop 内部的两个调度机制。LockBoost 基于经过验证的持锁者身份与在线反馈控制有界的时

间片延长；LockNest 将稳定资源域与为每把观测到的锁记录的 CPU 执行历史结合起来。

3. 我们在 ARM64、x86、RISC-V 与 VMware guest 上验证了端到端吞吐和尾延迟收益。消融实验表明，放置在纯 OLTP 中占主导，而在 CPU 干扰下，有界持锁者时间片延长能够提供额外收益，同时不减少批处理工作量。受控交接实验与处理器计数器进一步将这些运行区间与持锁者延迟和缓存局部性联系起来。

2 背景与动机

2.1 Linux 调度与竞争边界

Linux 使用 EEVDF 依据虚拟截止期与 lag 在合格的可运行任务中选择 [11]。它提供通用的公平性与延迟纪律，但一个可运行的持锁者与一个不相干的可运行任务是普通竞争者：调度器并不知道完成其中一个可能解除大量请求的阻塞。空闲 CPU 选择与负载均衡同样只看到任务亲和性与 CPU 拓扑，而不知道哪个唤醒者、等待者和执行位置属于同一把锁。

Linux 6.12 合入的 `sched_ext` 为 CPU 选择、enqueue、dispatch、running 与 stopping 提供 verifier 强制检查的 BPF 回调 [12]。BPF map 也可以由同步路径上的探针更新。这就在锁事件与下一次调度回调之间建立了直接的内核内通路：观测记录一把锁与其等待者、持有者和执行位置之间的关系，而 `select_cpu()` 与 `tick()` 使用这些信息，无需内核补丁或用户态策略往返。这里的机会不只是可编程性，还包括把短暂锁事件的身份保留到调度器需要采取动作的时刻。

目标负载中的大多数 mutex 获取始于用户态。一个无竞争的 pthread mutex 走完快速路径，对内核有意不可见。一旦快速路径失败，等待者会在普通、共享、requeue 或优先级继承（PI）futex 路径中阻塞 [13]。之后的唤醒使等待者变得可运行，调度器再决定其执行时间与 CPU。内核 mutex 与读写信号量（rwsem）具有类似的慢路径。因此竞争天然形成了一条观测边界：无竞争操作保持不受影响，而能从调度动作获益的事件则穿过一条内核可见的路径。

然而默认接口在这一边界丢失了结构。一次唤醒只说明任务可以运行。它没有指出哪个同步对象引起了唤醒、该任务是否会在下一次获得这把锁、哪个持锁者当前阻塞了它，也没有指出这把锁的持有者最近在哪里运行。第 3 节的逐锁记录正是用于重建这种结构。面向关键请求的 I/O 优先级机制也曾利用用户态/内核边界在阻塞路径上传递锁所有权 [14]。

2.2 可运行持锁者延迟、反复竞争与放置局部性

持锁者延迟会拉长锁占用。锁占用跨越从获取到释放的区间，而临界区只描述该区间内的应用工作。如果持锁者变得可运行、却排在不相干工作之后，调度延迟就会叠加到每个依赖等待者的阻塞时间上。当许多任务等待同一把锁时，一点点延迟的 CPU 服务就会变成巨大的聚合停顿。Lock-holder preemption 是这种更广义调度延迟的重要情形之一 [15, 9]。已有方案分别把运行时锁状态暴露给调度器、在 vCPU 抢占下调整自旋策略、用 eBPF 检测被抢占的临界区，或由应用显式请求临界区时间片扩展 [16-19]。

当许多等待者围绕同一把锁反复唤醒、获取失败并再次阻塞时，持锁者延迟会被进一步放大。虽然许多线程参与其中，但只有当前持锁者与少量后续任务能够推进串行化的请求路径。把所有可运行参与者当作独立负载，会在反复竞争上耗费 CPU，同时延迟真正改变锁状态的持锁者。这种现象推动了锁感知调度、竞争控制与数据库级准入机制的研究 [20-22]。

及时释放并不单独决定下一轮临界区的成本。后继任务通常会触碰前一位持锁者用过的锁字与被保护数据。相邻 CPU 之间的交接可以复用私有或共享缓存路径；跨 LLC、die 或 socket 边界的交接则需要额外的缓存一致性事务与缓存填充。因此相关拓扑是完整的缓存一致性层次，而非二元的 local/remote NUMA 标签。锁层设计通过层次化队列、调度感知的局部交接、按拓扑路由的次序或临界区委托来利用这一层次 [23-26]。

远端等待者所占比例越高，拓扑无感的选择就越可能沿昂贵的一致性路径转移所有权。反复交接会进一步放大成本，因为锁缓存行与被保护状态不断跟随后继任务迁移。LockNest 记住每把锁近期在哪里运行，并把这段历史限制在其资源域内；聚合负载均衡无法做出这种逐锁区分。

2.3 同一依赖的两个调度动作

持锁者推进与同锁放置解决同一条依赖的不同部分。推进持锁者改变 unlock 何时发生，却不选择缓存局部的后继任务；放置后继任务缩短下一次交接，却无法帮助一个被正确放置、但仍排在 CPU 干扰之后的持锁者。更好的放置能通过缩短临界区来减少后续的持锁者延迟，因此在一个给定负载中，一个动作可能消耗掉大部分机会。在混合 CPU 压力下，两种情况可能同时出现：交接流受益于局部性，而其当前持锁者仍需要及时服务。

两个动作的相对价值取决于负载，因此需要在受控锁竞争、端到端 OLTP 与 CPU 干扰下分别及组合评估它们。

设计目标是调度器机制，而非替换锁。LockCoop 保持 pthread/futex 协议、数据库 latch、内存放置与应用二进制不变。它观测自然穿过内核的竞争路径，把该依赖重建为有界状态，并在下一个决策点提供相应的调度动作。由此得到的抽象可以与锁层准入、队列排序与内存放置共存，因为它控制的是另一种资源：为被观测依赖提供的 CPU 时间与 CPU 位置。

3 LockCoop 设计

图 2 概括系统记录的关系，以及解锁前后的两个调度动作。

3.1 记录缺失的关系

观测与动作发生在不同上下文。锁探针能够识别等待者和持锁者，却不能为未来的唤醒选择 CPU；调度钩子能够在唤醒时行动，但单独的唤醒已经无法说明由哪把锁引起。一条有界记录只把缺失的身份跨越这段时间传递到调度点，而不要求服务器软件传播调度元数据。

两个调度动作仍必须指向同一把锁。聚合竞争信息无法判断哪个可运行任务正在延迟哪组等待者，而虚拟地址本身也不是进程间全局唯一的身份。因此，对于一次竞争事件，LockCoop 记录 $R = (w, K, o, c, t)$：等待

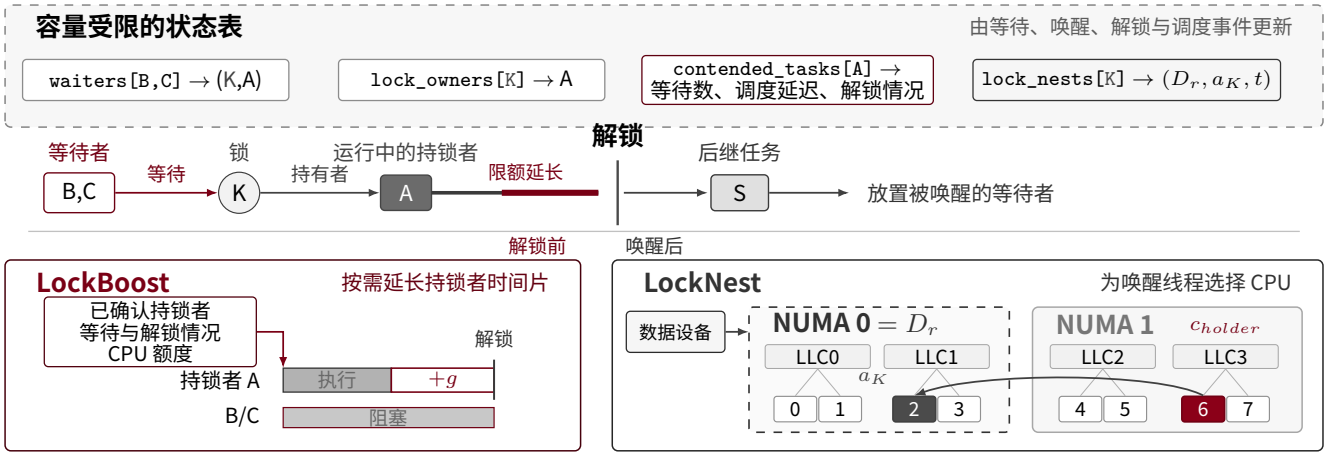


图 2 LockCoop 记录一把锁与其等待者、持有者和执行位置之间的关系，并在解锁前和唤醒后采取动作。

者 w、锁键 K、可选持锁者 o、观测 CPU c 与时间戳 t。这些记录把该依赖编码为调度器使用的有界状态。带作用域的锁键 K 区分锁类型与所有权域，将同一把锁的不同别名映射到同一个键，同时区分彼此无关的锁实例。

即使不知道持锁者，一次唤醒也可为后继放置提供证据；时间片扩展则必须先验证持锁者。“已验证”表示同步路径已经确认持锁者身份；第 4.1 节说明锁键构造与各路径的持锁者检查。因此，每个事件都注明它允许扩展、放置或两者。带有持锁者 TID 且通过验证的 mutex、PI、rwsem writer 与标注路径允许两个动作。Pthread rwlock 探针保留 reader/writer 语义，只有已验证的 writer 才允许扩展。共享 futex 与 requeue 等不含持锁者的交接证据只允许放置。持锁者证据与时间信息同锁键分开保存，使后续事件能够增强或过期证据而不改变锁身份。

观测发生在高频同步路径上，因此状态查找与回收必须保持有界。记录状态使用有界 map，它们只保存两个动作所需的字段；每个调度钩子只进行直接且有界的查找。核心依赖状态按生命周期分开：等待者状态只消费一次，持锁者状态在 unlock 时结束，而锚点历史需要跨越一次交接继续保留。使用同一清理规则，要么会保留陈旧的持锁者，要么会过早丢弃仍有价值的局部性历史。`waiters` 把 TID 映射到锁键与可选持锁者；`lock_owners` 把锁键映射到当前持锁者；`contended_tasks` 保存持锁者的等待任务数量、持锁者证据、可运行延迟与扩展记账；`lock_nests` 在容量 1,024 的 LRU 中保存近期同锁锚点、来源与时间信息。逐锁和逐 cgroup map 保存 LockBoost 的控制与预算状态（第 4.2 节）。map 丢失只移除一项优化，永远不会移除一个可运行任务。

身份与生命周期遵循不同规则。Acquire 替换持锁者状态；unlock 移除对应持锁者并刷新交接锚点；syscall 退出或 `select_cpu()` 原子地读取并清除等待者状态。锚点条目仅作为近期放置历史保留。因此，在同一地址空间内于相同虚拟地址重新初始化 mutex 可能复用键，但旧时间戳无法建立已验证持锁者，过期后即被拒绝。这在不把分配器代际逻辑放到观测热路径上的前提下，限制了陈旧放置的生命周期。

3.2 有界持锁者推进

锁竞争可能在持锁者已经进入运行状态后才变得可见。重排调度队列无法加速这样的持锁者；剩余的机会是保持它朝 unlock 推进的执行进展。因此，LockBoost 仅在已验证持锁者既被竞争、又被延迟时行动。这里的“推进持锁者”指延长正在运行持锁者的有界剩余时间片，而非在调度队列中提升它或改变其优先级。仅有等待任务数量无法判断持锁者是否被延迟，也无法判断近期服务是否释放了等待任务。因此，控制器对等待任务数量、持锁者从可运行到实际运行的调度延迟、以及扩展后的进展使用权重为 $1/8$ 的指数加权移动平均。这三类信号回答不同问题：等待任务数量估计有多少被阻塞任务可能受益，调度延迟判断持锁者是否确实被推迟，而扩展后等待任务的完成情况用于判断额外 CPU 时间是否缩短了等待。任何单一信号都不充分：快速临界区也可能积累许多等待任务，持锁者延迟也可能来自增加 CPU 时间无法修复的原因。

等待者阈值 T 初始为 4，并始终限制在 $[2, 16]$。每个控制窗口更新一次：当持锁者延迟超过 500 μs、至少有 T 个等待任务，并且额外服务后释放了等待任务或等待任务数量达到 $2T$ 时，阈值减 1，使控制器更易触发；若前一次动作被预算或长临界区守卫拒绝，或等待任务数量与延迟都较低，则阈值加 1；其他情况保持不变。反馈只改变扩展的触发难度，持锁者验证、服务预算与扩展上限仍是硬边界。候选持锁者只能获得短扩展；至少有 T 个等待任务的已验证持锁者获得常规有界扩展；较多等待任务与较高延迟可选择更强档位。unlock 间隔低于 62.5 μs 时抑制扩展，因为持锁者已经在快速推进。

单一预算无法同时约束任务、依赖链与工作负载三个范围内的干扰。任务 bucket 防止一个持锁者长期独占 CPU。锁 bucket 把限制延续到同一热点锁的后续持锁者。cgroup bucket 则约束一个工作负载中多把锁的总体影响。同时从三级 bucket 扣除额度，可防止持锁者更替或锁数量增加绕过局部限制。对每个 bucket i，经过的时间以速率 r_i 补充 token，最多达到 B_i；大小为 g 的扩展要求三个 bucket 都有足够余额：

$$b_i \leftarrow \min(B_i, b_i + r_i \Delta t), \qquad \text{允许扩展} \Leftrightarrow \min_i b_i \geq g. \tag{1}$$

获准的扩展从三个 bucket 同时扣除。自适应配置使用 100-ms 窗口、20/40/40% 的任务/锁/cgroup 预算、1 与 2-ms 的常规/强扩展、4-ms 累计时间片上限、以及 5-ms 连续执行守卫。unlock 撤销活跃状态；无等待者进展的连续执行进入冷却。无限时间片仅作为显式实验消融保留。

图 3 总结有界持锁者推进控制器及其撤销路径。

表 1 各类同步事件能够启用的调度动作。所有路径均通过独立的转换与拆除验证；性能评估聚焦于 mutex/futex 路径。

路径族	身份 scope	持锁者证据	启用的动作
内核 mutex/rwsem	内核对象	直接 owner/writer	推进 + 放置
私有 futex/pthread mutex	地址空间 + 地址	已验证 owner word	推进 + 放置
共享/PI futex	规范 futex key	PI owner 或交接	PI：两者；共享：放置
Futex requeue	源/目标 key	可选	放置
Pthread rwlock	地址空间 + 地址	已验证 writer / reader 集合	Writer：有界时间片延长
标注 spin/CAS	声明的 scope 与 key	可选 owner TID	由证据选择动作

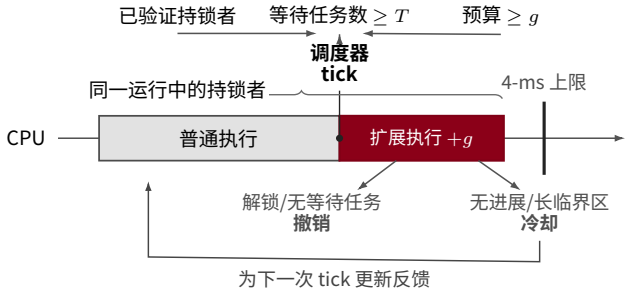


图 3 有界的持锁者时间片扩展。

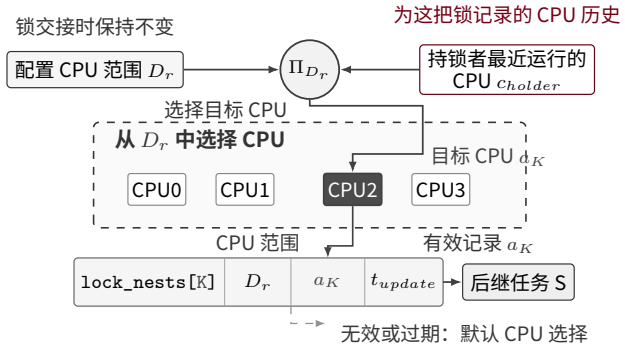


图 4 资源引导锚点的建立与使用。

3.3 资源引导的同锁放置

资源域稳定，但粒度太粗，不能保留单独一把锁的 CPU 和缓存历史。最近一位持锁者的 CPU 更精确，却可能属于偶然的资源域。LockNest 因此为每把锁记录两个位置：资源域 D_r，以及最近一位持锁者运行过的 CPU a_K，本文称 a_K 为该锁的锚点。加载器利用内核拓扑，将所配置数据设备（保存工作负载数据的块设备）所在 NUMA 节点的 CPU 定义为 D_r；若无法解析资源域，LockNest 仍为每把锁使用普通锚点路径。但在第一次交接之前，这样的锚点尚不存在。因此，每个新执行的用户任务会在资源域中放置一次。该一次性放置在逐锁锚点尚未建立、也无需等待时间窗口时，为后续锁执行提供域内的初始 CPU。稳态唤醒使用投影到同一域的锚点。两个阶段处理锚点生命周期中的不同位置：首次放置在锚点尚不存在时决定第一次锁执行的位置，投影则在持锁者更替后维持该资源域。只有首次放置无法阻止后续漂移；只有投影则必须在偶然的初始放置已经发生后再纠正它。其中，$\Pi_{D_r}(c)$ 将 CPU c 映射到 D_r 内对应的拓扑槽位，并同时满足在线与亲和性约束：

$$a_K \leftarrow \Pi_{D_r}(c_{holder}), \qquad c_{next} = a_K \text{ 若新鲜且被允许.} \tag{2}$$

投影保留同一把锁的持有者最近在缓存拓扑中的运行位置，也防止一个偶然的首位持有者把锁固定到不相干的 NUMA 域。离线 CPU、cpuset 违规、过期状态或被拒绝的锚点都回到普通 CPU 选择。

图 4 展示稳定资源域和为每把锁记录的 CPU 历史如何生成放置任务所用的锚点。

放置使用记录的执行历史，而不是当前等待者数量。唤醒事件确定等待者对应的锁后，有效锚点无需扫描或估计队列即可提供所需的 CPU 历史。等待者数量变化时，该锚点仍然有效。因此，选定策略使用 2-s 时间范围内的所有锚点；陈旧或非法锚点回退到普通 CPU 选择。第 5.3 节评估了更严格的接纳与生命周期配置，但都没有改善重复 OLTP 性能。

3.4 组合与回退

两个动作发生在 unlock 的两侧，并且需要不同强度的证据。组合调度器共享观测结果，但在相应钩子独立决定每个动作。统一的触发条件会丢弃仍然有用的部分证据：只有 wake 的路径可以在不知道持锁者时支持后继放置，而已验证且被延迟的持锁者即使没有有效锚点也可能需要时间片扩展。分别判断触发条件使每条依赖只触发其证据能够支持的动作。`tick()` 限制被竞争持锁者的剩余时间片；`select_cpu()` 读取并清除等待者状态并应用有效锚点。持锁者证据缺失会禁用时间片扩展，但不禁用放置；锚点状态陈旧会禁用放置，但不撤销有效的时间片扩展。若两个动作都不适用，任务使用与锁无感知 `sched_ext` 基线相同的虚拟时间路径。允许的 CPU mask 始终是允许放置任务的范围，且所有 BPF 程序在 attach 前都通过内核 verifier。

4 实现

4.1 观测路径

不同同步路径暴露不同的身份与所有权语义，而调度回调需要一个固定接口。LockCoop 因而由 BPF 锁事件程序、`sched_ext` 调度回调与一个小型用户态加载器组成。内核 mutex/rwsem 路径使用 kprobe 与 tracepoint；futex 路径捕获私有、共享、PI、wake 与 requeue 语义；uprobe 或标注 ABI 覆盖 pthread rwlock 与可选用户态同步。所有事件更新同一组逐锁记录，因此调度回调只读取锁身份和允许的动作，无需处理不同路径的解析细节。

身份归一化遵循各路径的语义：内核对象使用内核地址与全局作用域；私有用户锁把虚拟地址与地址空间身份组合起来；共享与 PI futex 使用 `get_futex_key()` 返回的规范身份。对于评估采用的 glibc/NPTL 环境，加载器根据 libc 布局获得 pthread-mutex owner-word 偏移，并在启动时使用普通、递归和错误检查 mutex 完成验证。候选 TID 只有在 task 查找与同地址空间检查之后才成为已验证持锁者。owner miss 仍形成放置证据，但不能授权

时间片扩展。PI 状态提供内核建立的持锁者，而 requeue 记录一个有界的源到目标重定向。Syscall 退出清理与原子等待者 claim 防止两条路径消费同一事件。

内核 mutex 入口/返回记录成功的所有权，慢路径在创建等待者依赖前解析当前持锁者。unlock 移除匹配的持锁者状态、刷新强锚点、并标记任何扩展以撤销。rwsem 观测保留一个 writer 持锁者与一个 reader 计数，而非把 reader 强行塞进单持锁者表示。pthread-rwlock uprobe 同样区分读获取、写获取、竞争与释放；只有已验证的 writer 所有权才启用面向持锁者的时间片延长。可选 spin/CAS ABI 通过同一事件接口导出 wait/acquire/release 事件与可选 owner TID。

4.2 调度钩子与有界状态

唤醒与 tick 都是延迟敏感的调度路径，因此策略决策避免拓扑遍历和无界搜索。`select_cpu()` 执行首放置、或消费等待者并测试锚点。`enqueue()` 记录持锁者可运行时间并按虚拟时间插入普通任务。持锁者开始执行后，`running()` 关闭其可运行延迟样本，并可赋予一次初始的短有界扩展；后续 `tick()` 可将剩余时间片延长到所选上限。两个钩子都不会将排队任务提前或重排调度队列。`stopping()` 对运行时间记账。每个调度决策使用常量次数的 map 操作；有界循环只扫描固定大小的拓扑数组。

加载器解析所配置数据设备的 NUMA 节点、填充 CPU/拓扑 map、配置控制器边界、attach 探针、然后注册调度器。detach 把任务全部交还给 EEVDF。状态 map 是建议性且有界的：分配失败、LRU 淘汰、不支持的事件或陈旧状态只增加诊断计数并选择回退路径。

owner、waiter 与 contention map 各持有 10,240 个条目；anchor 与 per-lock boost map 各 1,024；cgroup 预算状态 4,096。CPU 到节点、CPU 到 slot、节点到 CPU 数组由加载器一次性填充，因此锚点投影在唤醒时不遍历拓扑。每任务 BPF storage 携带一位首放置 token，并由 `sched_process_exec` 重置。

4.3 加载与失败语义

独立测试覆盖表 1 中的每类同步路径，检查私有映射和共享映射下的锁身份、持锁者验证及状态清理。其他测试覆盖地址复用、futex 等待队列转移、读者与写者动作的区分，以及 LRU 淘汰。这些测试均未出现验证器失败、map 更新失败、任务停滞或无法继续执行的情况。

5 评估

我们评估随着工作线程数量增加、等待锁的情况变得频繁，LockCoop 能否维持吞吐并限制尾延迟增长。受控 pthread-mutex 实验首先隔离持锁者时间片延长与后继任务放置，随后用策略实验评估所选边界与在线控制。MariaDB/Sysbench 将客户端数量从 64 增加到 256，用于测量并发数达到或超过 CPU 数量时的端到端行为。同时运行的 CPU 密集型后台任务用于判断持锁者时间片延长何时能在放置之上产生额外收益。随后，我们在固定 CPU 竞争下测试 MariaDB、PostgreSQL、SQLite 和 RocksDB，以考察不同应用中的效果。最后，我们在多种处理器架构上重复 OLTP，并与锁层和内存层机制比较，以评估可移植性与机制层次差异。

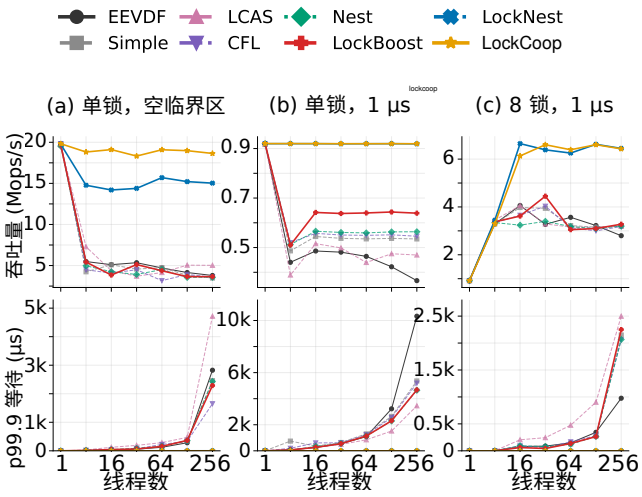


图 5 Mutex 动作隔离。上：吞吐；下：一把热点 mutex、八把热点 mutex 与空临界区的 p99.9 锁等待。

5.1 实验设置

主负载是 Sysbench OLTP read/write，包含 16 张表、每表一百万行。性能实验重点评估会使用 mutex 和 futex 路径的负载。Linux default 与所有 `sched_ext` 用例统一使用 `performance` governor。我们对比 stock EEVDF、上游锁无感知 `scx_simple`（图中记为 Simple）、依据已发表 LCAS、CFL 与 Nest 策略完成的 `sched_ext` 移植、两个动作消融 LockBoost/LockNest，以及组合 LockCoop 。这些移植实现执行相应论文中的策略决策，并在全部正式测量中使用同一组配置：LCAS 在 1 秒窗口内采用 10/6% 的竞争滞回阈值；CFL 采用 4 个 waiter 的阈值、200 ms 衰减时间和 1 ms owner credit；Nest 采用 $r_{\max}/r_{\text{impatient}} = 5/2$、2 ms 移除时间和 20 ms 时间片 [6-8]。“固定”仅指这些外部配置参数；各策略的在线滞回、衰减、credit 与放置决策仍然正常运行。除非另有说明，每个 OLTP 配置重复十次，图中报告均值与 95% 置信区间。

Sysbench 以共置的闭式 OLTP 工作负载驱动 MariaDB。所报告的吞吐衡量端到端完成事务数。“不绑定”表示未施加外部 CPU 或内存亲和性；包括 LockCoop 资源域选择在内的调度器内部放置策略仍然生效。正式 OLTP 点使用 10-s warmup 加 60-s measurement。

应用范围实验在 x86 机器上以全机不绑定方式运行，在 128 和 256 个工作线程下比较 EEVDF、`scx_simple` 和 LockCoop 。负载包括使用 Sysbench OLTP read/write 的 MariaDB 和 PostgreSQL、采用 WAL 模式并执行 90% 读取和 10% 更新的 SQLite，以及运行 YCSB Workload A 的 RocksDB。PostgreSQL 使用 `synchronous_commit=off`。每个前台负载同时运行一个 4 线程 Fluidanimate 实例和一个 10 线程 Streamcluster 实例。每组配置预热 30 秒、测量 120 秒，并按随机顺序重复三轮。

5.2 受控 Mutex 交接

动作隔离。 我们首先变化锁数量与临界区工作：一把 1-μs 临界区 mutex、八把 1-μs 临界区 mutex、以及一把空临界区 mutex。图 5 报告到 256 线程的吞吐与 p99.9 获取等待。

结果清楚地区分了两个调度机会。256 线程时，空临界区单锁场景下 LockCoop 达到 18.64 Mops/s，是

表 2 评估平台。

平台	处理器与被评估 CPU 范围	内存/存储	软件
ARM64 裸机	2× Ampere Altra Max M128-30；256 核，无 SMT；节点 0 或不绑定	1.0 TiB；Intel NVMe	openEuler 24.03；Linux 6.17；MariaDB 10.6.27
x86 裸机	2× Xeon Platinum 8375C；64 核/128 线程；节点 0 或不绑定	256 GiB；Intel NVMe	Ubuntu 24.04；Linux 6.17；MariaDB 10.11.14
RISC-V 裸机	Milk-V Pioneer；64 个 C920 核；4 个 NUMA 节点；不绑定	128 GiB；NVMe	RevyOS；Linux 6.17.13；MariaDB 11.8.6
VMware guest	32 vCPU；单 vNUMA 节点；VMware 暴露的 EPYC 9654 CPUID	128 GB；120 GB PVSCSI	Ubuntu 26.04；Linux 7.0；MariaDB 11.8.6

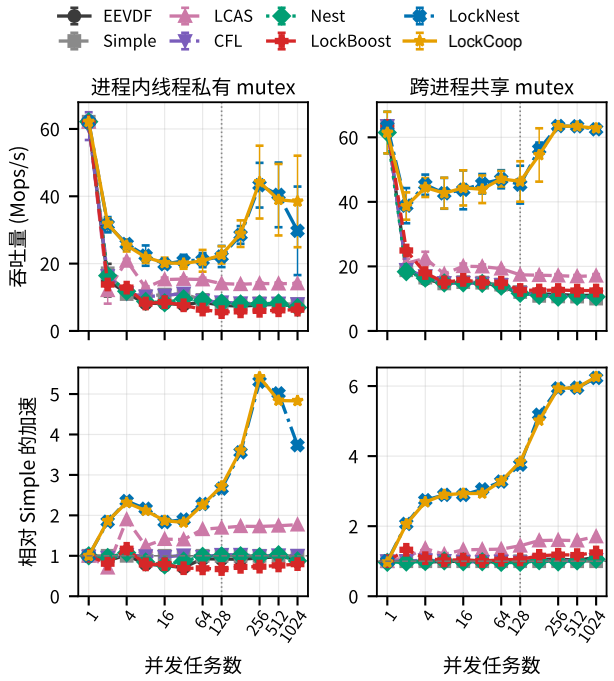


图 6 受控 mutex 交接。上：吞吐；下：相对 Simple 归一化吞吐。虚线标记 128 个在线 CPU。

`scx_simple` 的 5.36 倍，同时把 p99.9 等待保持在测量下界；八把独立 mutex 时，LockNest 达到 EEVDF 的 2.30 倍。因此，一条串行依赖链突出持锁者扩展的作用，多条独立依赖链则保留逐锁放置收益。

线程域与进程域。 线程私有内核在多线程间共享一个静态 pthread mutex；进程共享内核在共享映射中使用 `PTHREAD_PROCESS_SHARED` 自适应 mutex。每个临界区递增一个计数器。我们在 x86 上不绑定地 sweep 1–1024 个任务；图 6报告十次运行均值。

两种 mutex 作用域在超售下都保持收益。1024 线程时，LockCoop 是 LockNest 的 1.29 倍；1024 进程时，它是 `scx_simple` 的 6.26 倍。独立 LockBoost 达到 `scx_simple` 的 1.25 倍，说明持锁者扩展确有帮助，但这些短临界区仍主要受益于后继放置。

调度器计数区分有用进展与无差别扩展。进程共享内核中，24.3% 的采样 LockBoost 持锁者处于可运行状态，而线程私有内核中为 0.64%。组合保留 89.1% 的局部性命中率，同时把扩展降到每百万次操作 13.34 次：稳定放置使需要 LockBoost 推进的延迟持锁者变少。

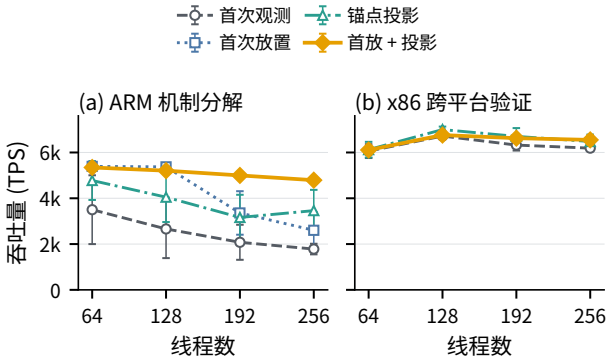


图 7 全机不绑定条件下的资源引导 LockNest。ARM 分解首次放置与锚点投影，x86 评估完整策略；误差线为 95% 置信区间。

5.3 策略刻画

图 7 把资源引导放置分成两部分：每个任务执行一次首次放置，以及把每个锚点稳定地投影到 D_r，实验不施加外部 CPU 或内存绑定。ARM 分别开关两个动作，x86 则在第二种体系结构上评估完整策略。所配置的数据设备在 ARM 上解析到节点 0，在 x86 上解析到节点 1。

ARM 上 256 线程时，完整策略达到首次观测锚定的 2.69 倍，并将多次运行之间的变异系数从 18.19% 降到 4.41%。首次放置或锚点投影单独启用时都无法达到这种稳定性：前者为后续锚点播种资源域，后者把之后的同锁执行限制在该域内。x86 上，锚点投影贡献了主要收益；加入首次放置后，整个并发区间内与仅投影相差不超过 3.4%，并在 256 线程达到 6,550.1 TPS。因此，同一策略无需平台专属锚点规则即可保持 x86 的扩展趋势。

实验性的等待者压力阈值 $Q = 8$–128 会减少逐锁放置，但没有一种配置在正式重复中始终超过直接使用所有有效锚点。1–64 热点锁压力测试无 map 失败或零进展线程，因此选定策略在常见唤醒路径上不计算队列长度。

图 8 把自适应控制与静态有界、无界扩展分开。自适应控制在重复 OLTP 中优于静态有界控制，并在同时运行 cyclictest 时把其 p99/p999 延迟限制在 175/363 μs。2-ms 临界区下其 Jain 指数为 0.9805，而无界消融为 0.8906。在零工作快速锁上，自适应控制与不启用扩展相差不超过 0.99%，而静态有界控制损失 14.6%。因此，在线证据决定有限扩展何时有用，而非简单地加长时间片。

在一次持续 20 秒的 MariaDB 诊断实验中，只有 1.9% 的持锁者调度延迟样本对应时间片扩展。几乎所有扩展

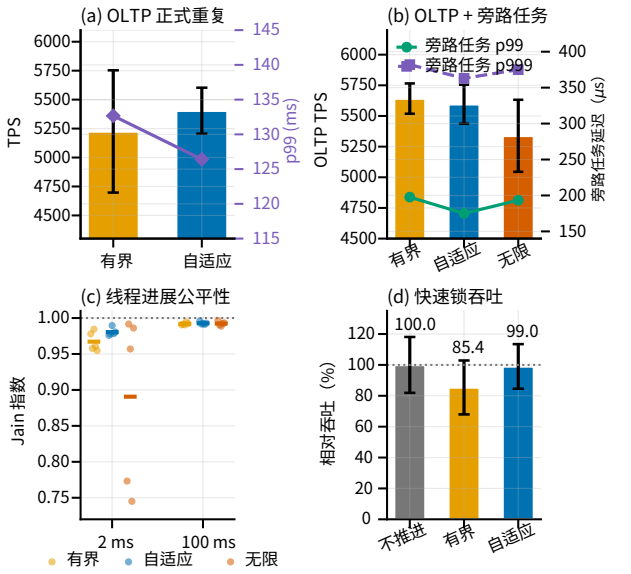


图 8 有界持锁者推进控制：重复 OLTP、混合负载中受干扰任务的延迟、临界区公平性与短临界区保护实验。

都在解锁时撤销。在另一项快速锁诊断实验中，自适应控制检测到持锁者已能快速推进后，不再发放扩展。

参数研究覆盖 $T = 2$–16、250–2,000-μs 扩展、1–10-ms 上限与 5–20% 任务预算。没有任何单一合成最优点决定选定配置：重复 OLTP、混合负载中受干扰任务的延迟、长临界区公平性与短临界区保护共同划定运行区域。控制器随后在这些边界内在线选择动作。

5.4 端到端性能与机制证据

图 9 在五个执行范围内展示一致的高吞吐结果。256 线程时，`scx_lockcoop` 相对 `scx_simple` 提升 2.15–13.9 倍吞吐、降低 56.6–94.0% 的 p95。代表性的 ARM64 节点 0 结果为 5.30 倍吞吐与 84.8% 的 p95 降幅，x86 不绑定则达到 4.14 倍与 82.2%。在 64 核 RISC-V 上，`scx_lockcoop` 达到 2.15 倍吞吐并将 p95 降低 56.6%。不绑定实验仍保持高吞吐，32-vCPU VMware guest 达到 `scx_simple` 的 4.42 倍。

在相同节点 0 测试点上，`scx_lockcoop` 分别比 ARM64 和 x86 上更强的 LCAS/CFL 结果高 5.16 倍和 2.61 倍。聚合竞争引导在部分场景中有效，但缺少逐锁交接身份时无法进入相同的高吞吐区间。

ARM64 节点 0 的 64 线程下，各机制因竞争较弱而收敛；随着等待任务数量增长，LockCoop 的优势才逐渐出现。尽管三种裸机 ISA 的核心、SMT 与拓扑组织不同，同样的定性转变都会出现。

图 10 在受控 CPU 干扰下评估持锁者推进何时能够在同锁放置之上产生增量。

纯 OLTP 由局部性主导：256 线程时，LockNest 与 LockCoop 在 ARM64 节点 0 上相差 1% 以内、在 x86 节点 0 上相差 0.8% 以内，而独立 LockBoost 仍位于低吞吐区间。这一消融表明，此时放置已经捕获几乎全部收益。为在同一条端到端路径上暴露持锁者延迟，我们保持 256 线程 x86 不绑定前台不变，并引入 2–16 个固定时长的 CPU 密集型后台任务。每次 240 秒运行依次包含 60 秒纯前台、120 秒并发批处理和 60 秒恢复阶段。

LockCoop 在干扰区间之外与 LockNest 基本一致，并在干扰结束后恢复。干扰期间，其平均吞吐优势随后台任务从 2 个增加到 16 个，由 9.6% 增长到 36.1%，其中四个后台任务点为 18.2%。两个调度器下的批处理工作量差异不超过 0.53%，后台任务 CPU 时间也保持一致。独立 LockBoost 曲线仍位于低吞吐区间；此时有界持锁者推进通过保护已被 LockNest 局部化的依赖链产生增量。

接下来，我们评估 CPU 竞争下的结果是否只存在于 MariaDB。图 11 在相同的后台计算负载下比较四种存储系统。

在 128 和 256 个工作线程下，LockCoop 在四种应用中的吞吐量均高于 Simple，达到其 1.11–3.51 倍。

图 12 评估后继放置如何改变 MariaDB 的处理器执行行为。256 线程时，LockCoop 把 IPC 从 0.63 提高到 0.99、把迁移减少 62%，同时保持相近的数据库忙碌 CPU 数；LockNest 独立达到 IPC 1.00。LockCoop 的缓存未命中率保持在 6–7%，也低于通用拓扑 Nest 的 20.3%。结合 2×2 分解（图 7），相近的忙碌 CPU 数量排除了额外 CPU 容量。相对 Simple，更低的迁移率、更高的 IPC 和更少的缓存未命中表明，LockCoop 有效复用了为每把锁记录的 CPU 历史。

为确定高竞争区间中 MariaDB 的 CPU 时间主要消耗在哪里，图 13 展示 x86 全机不绑定的 256-client 测试点上 EEVDF 与 LockCoop 的 MariaDB 进程级 CPU 采样。

在这组 MariaDB 负载中，吞吐量提高的同时，每笔事务的 CPU 开销也有所下降。在 EEVDF 下，MariaDB 约一半的采样周期消耗在内核队列自旋锁慢路径，主要来自 futex 等待和唤醒。LockCoop 将这一占比降至 11.4%。六组配对计数器实验也显示，每笔完成事务所需的 CPU 周期更少。

5.5 跨层对比与快速路径成本

图 14 比较 LockCoop 与 CNA 的 NUMA 感知队列交接 [5]、GCR 的 active-set 准入及其 NUMA 扩展 [1]、以及 Linux AutoNUMA 放置 [27]。AutoNUMA 使用上游内核实现。在 EEVDF 下，我们的 pthread 兼容 CNA 移植实现已发表的本地 successor 与 secondary queue 交接策略，并采用自旋 2,048 次后 park 的 pthread 集成方式；GCR 与 GCR-NUMA 移植实现已发表的 active-set 准入策略，active 上限为 4、join 阈值为 2、批准周期为 16,384 次获取。所有 pthread 后端均通过 mutex 与条件变量语义测试。配套验证运行开启机制路径计数，确认 CNA 本地/远程交接、GCR 主动/被动准入及 GCR-NUMA 优选节点轮转实际执行；正式性能运行关闭这些计数器。LockCoop 保留 native pthread mutex，所有用例使用相同的 x86 放置范围与频率策略。

256 线程时，LockCoop 在节点 0 上达到 GCR 的 2.87 倍，在不绑定范围内达到 native pthread 的 3.08 倍；其节点 0 p99 为 89.16 ms，而 GCR 为 333.92 ms。GCR-NUMA 仅在不绑定时比 GCR 提高 1.17 倍，CNA 在两个范围内都低于 native pthread，分别为其 0.32 倍和 0.50 倍。在所测配置中，仅采用 NUMA 感知的锁获取顺序，并未达到 LockCoop 的收益。切换 AutoNUMA 使吞吐最多变化 4.72%、p99 最多变化 3.54%；LockCoop 在两

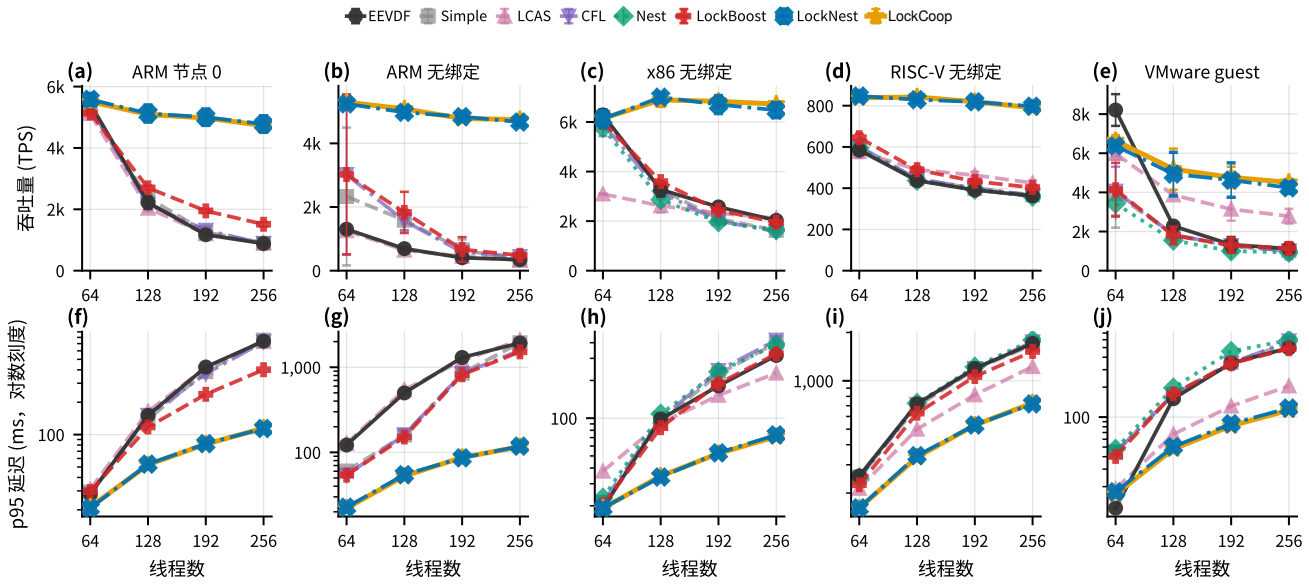


图 9 MariaDB/Sysbench OLTP 吞吐（上）与 p95 延迟（下）。

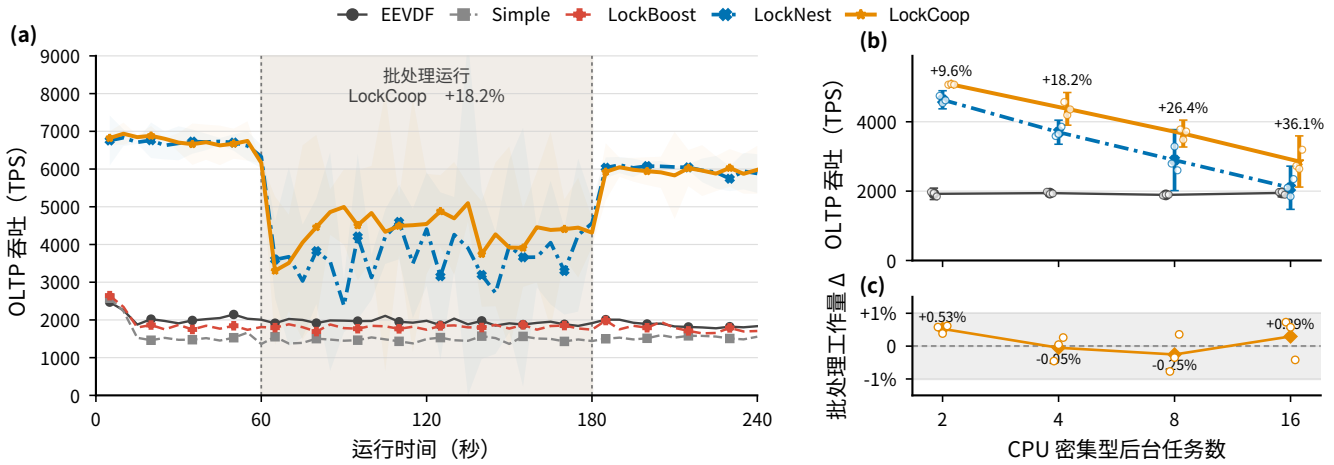


图 10 CPU 干扰下的 x86 不绑定 OLTP。(a) 四个后台任务时间线；(b) 2–16 个后台任务下干扰阶段的 OLTP 吞吐，空心点为单次结果，误差棒为十次运行的 95% 置信区间；(c) LockCoop 相对 LockNest 的逐次批处理工作量，阴影标出 ±1% 区间。

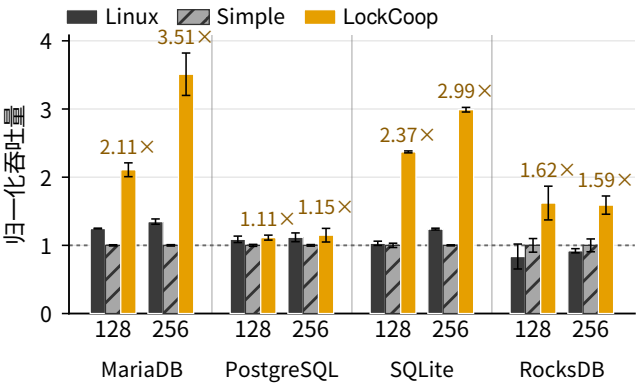


图 11 固定 CPU 竞争下的应用吞吐量，相对 Simple 归一化（均值 ± 标准差，三轮）。

种设置下都保持高吞吐。这些机制分别作用于锁队列顺序、准入并发、页面放置和逐锁 CPU 调度。

固定速率下，独立的定时器任务在完整配置下记录 59.98/74.65 μs 的 p99/p99.9 延迟。

端到端观测开销。 为隔离观测与调度动作，我们保留全部锁事件采集和状态维护，同时关闭 LockBoost 与

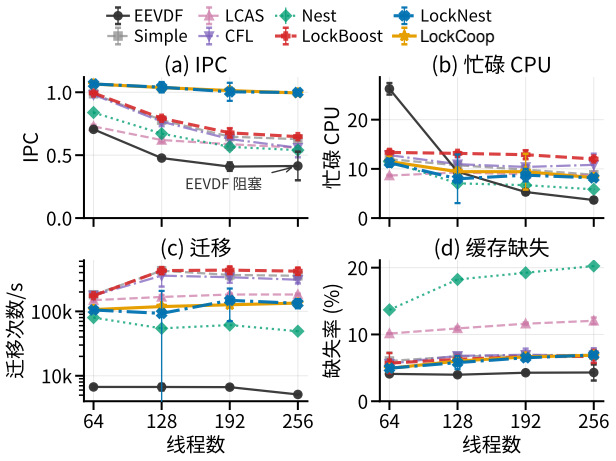


图 12 x86 MariaDB 进程计数器：IPC、忙碌逻辑 CPU、迁移数与 L1/L3 缓存未命中率。

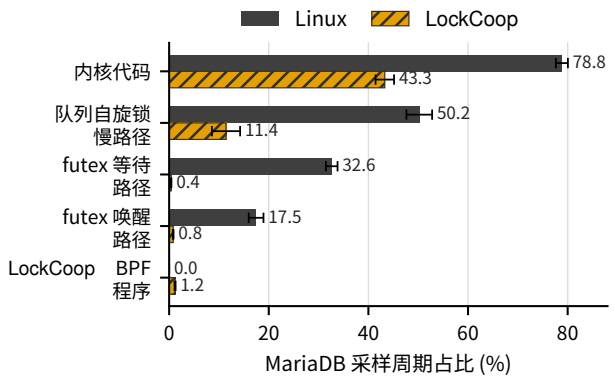


图 13 EEVDF 与 LockCoop 下 MariaDB 在 256 clients 时的 CPU 采样（三轮均值 ± 标准差）。各柱独立统计不同调用路径的周期占比，路径之间可能重叠。

表 3 快速路径压力结果。固定速率运行目标 0.50 百万次 futex 操作/秒；不限速运行报告持续速率。

Profile / key 集	速率 (Mops/s)	诊断
Scheduler-only / fixed	0.50	0.36 CPU-eq.
Full / fixed	0.50	1.16 CPU-eq.
Full / 单 key	4.41	0 失败/停滞
Full / 8,192 key	6.77	0 失败/停滞

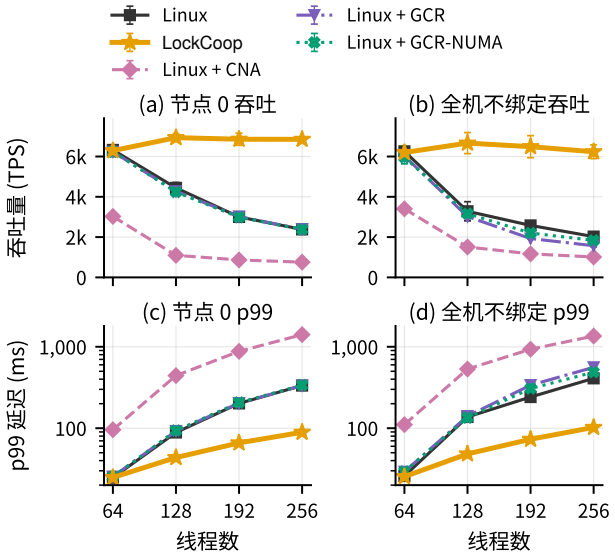


图 14 使用 native pthread mutex 的 x86 OLTP 吞吐与 p99；误差线为 95% 置信区间。

LockNest，并采用与 `scx_simple` 相同的虚拟时间策略。在十次 60 秒实验中，仅观测配置在 64 客户端下相对 `scx_simple` 的吞吐变化为 $+1.1\% \pm 1.2\%$（配对 95% 置信区间），表明其在非过载区间没有可测量的端到端性能退化。在 256 客户端下，其吞吐提高 $19.4\% \pm 1.2\%$。由于该过载运行点对微小的执行时序扰动较为敏感，我们仅将这一正向变化视为无退化结果，而不将其解释为观测机制带来的收益。完整 LockCoop 的提升无法仅由观测过程解释。表 3 进一步在固定事件速率下隔离了观测路径的处理成本。

5.6 运行区间与可移植性

在纯 MariaDB OLTP 和临界区最短的 mutex 测试中，后继放置贡献了几乎全部吞吐量收益。在 CPU 干扰和进程共享 mutex 测试中，持锁者时间片扩展也能带来收益。LockCoop 分别检查每种动作所需的证据，而不是为整个负载选定一种模式。

由于 D_r 不必对应一个 socket，且 a_K 记录域内 CPU，同一规则可以在节点受限时利用私有缓存与 LLC 局部性、在不绑定时利用 NUMA 局部性、并在 guest 内利用虚拟 CPU 局部性。不同平台改变的是拓扑与软件版本，而不是这条调度规则。

6 相关工作

锁感知调度与同步。 CA-Scheduler 使用锁状态调整执行，LCAS 把锁密集任务路由到专门核心，CFL 把锁占用作为资源调度 [16, 6-7]。Scheduler-conscious synchronization、SCL、SynCord、I-Spinlock 与 FlexGuard 通过锁、运行时或 vCPU 状态协调调度；RSEQ 允许经过插桩的用户态代码请求有界的临界区扩展 [15, 9-10, 17-19]。面向请求的 I/O 优先级机制也会传递用户锁持有者以继承 I/O 关键性 [14]。LockCoop 从现有路径推导每把锁的等待者与持锁者关系，并让锁身份贯穿持锁者服务与后继放置。数据库调度器与锁位点控制减少到达热点锁的工作，或选择专门锁实现 [28-30, 22, 31]；这些决策与两个调度动作互补。

锁算法与竞争控制。 MCS、Cohort、HMCS、shuffling、CNA、CST 与 RON 在锁协议内部组织等待者或锁转移，以控制竞争和局部性 [2-3, 23, 4, 32, 5, 24-25]。TCLOCKS 把临界区委托到特定核心执行，使被保护数据保持就近 [26]。GCR 与 HTLL 调节活跃竞争者，数据库控制则限制进入热点锁的工作 [1, 33, 28-29, 22]。LockCoop 保持准入与锁语义不变，调度被观测的持锁者与被唤醒任务。这些方法说明锁交接局部性能够减少一致性流量；LockNest 在锁外利用逐锁身份应用这一原则。

NUMA 与缓存感知调度。 Nest 是最接近的调度器层放置 baseline；它使用聚合复用信号，而不是某把锁自身的执行历史 [8]。缓存感知聚类同样使用聚合信号，NUMA 放置与 AutoNUMA 则把任务与内存共置 [34-35, 27]。近期 Linux 工作把共享地址空间的线程聚集到偏好的 LLC [36]；其它调度器使用应用、交互或虚拟拓扑信号 [37-40]。LockCoop 则识别每个同步信号对应的锁，并把该锁的 CPU 历史从私有缓存投影到 NUMA 域。

可编程调度器。 ghOSt 把策略委托给用户态，相关系统则为专用运行时提供机制 [41-47]。`sched_ext` 提供本文使用的内核 BPF 钩子 [12]；本文贡献是逐锁关系与两个有界动作，而不是调度器可编程性本身。

7 结论

LockCoop 记录每把锁的等待者、持有者与执行位置，在解锁前推进被延迟的持锁者，并在唤醒后保持后继任务的局部性。两个动作都能独立发挥作用，并在 CPU 干扰下相互补充；缺少支持证据时，任务仍使用普通调度。

参考文献

[1] DICE D, KOGAN A. Avoiding scalability collapse by restricting concurrency[C/OL]//Lecture Notes in Computer Science: volume 11725 Proceedings of the 25th International European Conference on Parallel and Distributed Computing (Euro-Par). Springer, 2019: 363-376. DOI: 10.1007/978-3-030-29400-7_26.

[2] MELLOR-CRUMMEY J M, SCOTT M L. Algorithms for scalable synchronization on shared-memory multiprocessors[J/OL]. ACM Transactions on Computer Systems, 1991, 9(1): 21-65. DOI: 10.1145/103727.103729.

[3] DICE D, MARATHE V J, SHAVIT N. Lock cohorting: A general technique for designing NUMA locks[C/OL]//Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP). ACM, 2012: 247-256. DOI: 10.1145/2145816.2145848.

[4] KASHYAP S, CALCIU I, CHENG X, et al. Scalable and practical locking with shuffling[C/OL]//Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP). ACM, 2019: 586-599. DOI: 10.1145/3341301.3359629.

[5] DICE D, KOGAN A. Compact NUMA-aware locks[C/OL]//Proceedings of the 14th European Conference on Computer Systems (EuroSys). ACM, 2019: 1-15. DOI: 10.1145/3302424.3303984.

[6] CUI Y, WANG Y, CHEN Y, et al. Lock-contention-aware scheduler: A scalable and energy-efficient method for addressing scalability collapse on multicore systems[J/OL]. ACM Transactions on Architecture and Code Optimization, 2013, 9(4). DOI: 10.1145/2400682.2400703.

[7] PARK J, EOM Y I. Locks as a resource: Fairly scheduling lock occupation with CFL[C/OL]//Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP). ACM, 2024: 17-29. DOI: 10.1145/3627535.3638477.

[8] LAWALL J, CHHAYA-SHAILESH H, LOZI J P, et al. OS scheduling with nest: Keeping tasks close together on warm cores[C/OL]//Proceedings of the 17th European Conference on Computer Systems (EuroSys). ACM, 2022: 368-383. DOI: 10.1145/3492321.3519585.

[9] PATEL Y, YANG L, ARULRAJ L, et al. Avoiding scheduler subversion using scheduler-cooperative locks[C/OL]//EuroSys '20: Proceedings of the Fifteenth European Conference on Computer Systems. ACM, 2020. DOI: 10.1145/3342195.3387521.

[10] PARK S, ZHOU D, QIAN Y, et al. Application-informed kernel synchronization primitives[C/OL]//Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2022: 667-682. https://www.usenix.org/conference/osdi22/presentation/park.

[11] STOICA I, ABDEL-WAHAB H. Earliest eligible virtual deadline first: A flexible and accurate mechanism for proportional share resource allocation: 95-22[R/OL]. Old Dominion University, 1995. https://www.cs.cmu.edu/~istoica/pubs.html.

[12] Linux kernel documentation. Extensible scheduler class[EB/OL]. 2024. https://docs.kernel.org/scheduler/sched-ext.html.

[13] FRANKE H, RUSSELL R, KIRKWOOD M. Fuss, futexes and furwocks: Fast userlevel locking in Linux[C/OL]//Proceedings of the Ottawa Linux Symposium. 2002: 479-495. https://www.kernel.org/doc/ols/2002/ols2002-pages-479-495.pdf.

[14] KIM S, KIM H, LEE J, et al. Enlightening the I/O path: A holistic approach for application performance[C/OL]//Proceedings of the 15th USENIX Conference on File and Storage Technologies (FAST). USENIX Association, 2017: 345-358. https://www.usenix.org/conference/fast17/technical-sessions/presentation/kim-sangwook.

[15] KONTOTHANASSIS L I, WISNIEWSKI R W, SCOTT M L. Scheduler-conscious synchronization[J/OL]. ACM Transactions on Computer Systems, 1997, 15(1): 3-40. DOI: 10.1145/244764.244765.

[16] XIAN F, SRISA-AN W, JIANG H. Contention-aware scheduler: Unlocking execution parallelism in multithreaded Java programs[C/OL]//Proceedings of the 23rd ACM SIGPLAN Conference on Object-Oriented Programming Systems Languages and Applications (OOPSLA). ACM, 2008: 163-180. DOI: 10.1145/1449764.1449778.

[17] TEABE B, NITU V, TCHANA A, et al. The lock holder and the lock waiter pre-emption problems: Nip them in the bud using informed spinlocks (I-Spinlock)[C/OL]//Proceedings of the 12th European Conference on Computer Systems (EuroSys). ACM, 2017: 286-297. DOI: 10.1145/3064176.3064180.

[18] LAFORET V, KASHYAP S, IORGULESCU C, et al. FlexGuard: Fast mutual exclusion independent of subscription[C/OL]//Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP). ACM, 2025: 1115-1130. DOI: 10.1145/3731569.3764852.

[19] Linux kernel documentation. Restartable sequences[EB/OL]. 2026. https://docs.kernel.org/userspace-api/rseq.html#scheduler-time-slice-extensions.

[20] LOZI J P, LEPERS B, FUNSTON J, et al. The Linux scheduler: A decade of wasted cores[C/OL]//Proceedings of the 11th European Conference on Computer Systems (EuroSys). ACM, 2016: 1-16. DOI: 10.1145/2901318.2901326.

[21] ISSTAIF A A T, KALYVIANAKI E, MORTIER R. Mitigating context switching in densely packed Linux clusters with latency-aware group scheduling[EB/OL]. 2025. https://arxiv.org/abs/2508.15703.

[22] CAI M, LIU S, HUANG H. tScale: A contention-aware multithreaded framework for multicore multiprocessor systems[C/OL]//Proceedings of the IEEE International Conference on Parallel and Distributed Systems (ICPADS). IEEE, 2017: 334-343. DOI: 10.1109/ICPADS.2017.00052.

[23] CHABBI M, FAGAN M W, MELLOR-CRUMMEY J M. High performance locks for multi-level NUMA systems[C/OL]//Proceedings of the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP). ACM, 2015: 215-226. DOI: 10.1145/2688500.2688503.

[24] KASHYAP S, MIN C, KIM T. Scalable NUMA-aware blocking synchronization primitives[C/OL]//Proceedings of the 2017 USENIX Annual Technical Conference (ATC). USENIX Association, 2017: 603-615. https://www.usenix.org/conference/atc17/technical-sessions/presentation/kashyap.

[25] LO S, LIN H T, HSIEH Y H, et al. RON: One-way circular shortest routing to achieve efficient and bounded-waiting spinlocks[C/OL]//Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2023: 17-31. https://www.usenix.org/conference/osdi23/presentation/lo.

[26] GUPTA V, DWIVEDI K K, KOTHARI Y, et al. Ship your critical section, not your data: Enabling transparent delegation with TCLOCKS [C/OL]//Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2023: 1-16. https://www.usenix.org/conference/osdi23/presentation/gupta.

[27] CORBET J. AutoNUMA: the other approach to NUMA scheduling [EB/OL]. 2012. https://lwn.net/Articles/488709/.

[28] TIAN B, HUANG J, MOZAFARI B, et al. Contention-aware lock scheduling for transactional databases[J/OL]. Proceedings of the VLDB Endowment, 2018, 11(5): 648-662. DOI: 10.1145/3187009.3177740.

[29] KALIKAR S, NASRE R. Toggle: Contention-aware task scheduler for concurrent hierarchical operations[C/OL]//Lecture Notes in Computer Science: volume 11725 Proceedings of the 25th International European Conference on Parallel and Distributed Computing (Euro-Par). Springer, 2019: 142-155. DOI: 10.1007/978-3-030-29400-7_11.

[30] CUI Y, WANG Y, CHEN Y, et al. Requester-based spin lock: A scalable and energy efficient locking scheme on multicore systems [J/OL]. IEEE Transactions on Computers, 2015, 64(1): 166-179. DOI: 10.1109/TC.2013.196.

[31] JI J, GU Y, FU Y, et al. Beyond the bottleneck: Enhancing high-concurrency systems with lock tuning[C/OL]//Proceedings of the International Symposium on Formal Methods. Springer, 2024: 325-337. DOI: 10.1007/978-3-031-71177-0_20.

[32] PUSUKURI K K, GUPTA R, BHUYAN L N. Shuffling: A framework for lock contention aware thread scheduling for multicore multiprocessor systems[C/OL]//Proceedings of the 23rd International Conference on Parallel Architectures and Compilation Techniques (PACT). ACM, 2014: 289-300. DOI: 10.1145/2628071.2628074.

[33] YU Z, GU J, WU Z, et al. HTLL: Latency-aware scalable blocking mutex[J/OL]. IEEE Transactions on Parallel and Distributed Systems, 2025, 36(3): 471-486. DOI: 10.1109/TPDS.2025.3526859.

[34] TAM D, AZIMI R, STUMM M. Thread clustering: Sharing-aware scheduling on SMP-CMP-SMT multiprocessors[C/OL]//Proceedings of the 2nd ACM SIGOPS/EuroSys European Conference on Computer Systems (EuroSys). ACM, 2007: 47-58. DOI: 10.1145/1272996.1273004.

[35] LEPERS B, QUéMA V, FEDOROVA A. Thread and memory placement on NUMA systems: Asymmetry matters[C/OL]//Proceedings of the 2015 USENIX Annual Technical Conference (ATC). USENIX Association, 2015: 277-289. https://www.usenix.org/conference/atc15/technical-session/presentation/lepers.

[36] CHEN T, CHEN Y. Cache aware scheduling[EB/OL]. 2026. https://lore.kernel.org/lkml/cover.1770760558.git.tim.c.chen@linux.intel.com/.

[37] TANG W, KE Y, FU S, et al. Demeter: QoS-aware CPU scheduling to reduce power consumption of multiple black-box workloads[C/OL]//Proceedings of the ACM Symposium on Cloud Computing (SoCC). ACM, 2022: 31-46. DOI: 10.1145/3542929.3563476.

[38] ZHENG Y, LI C, XIONG Y, et al. iAware: Interaction aware task scheduling for reducing resource contention in mobile systems[J/OL]. ACM Transactions on Embedded Computing Systems, 2023, 22(5s): 1-24. DOI: 10.1145/3609391.

[39] GUO E, JIA W, DING X, et al. Optimizing task scheduling in cloud VMs with accurate vCPU abstraction[C/OL]//Proceedings of the European Conference on Computer Systems (EuroSys). ACM, 2025: 753-768. DOI: 10.1145/3689031.3696092.

[40] ÁLVAREZ D, SALA K, BELTRAN V. nOS-V: Co-executing HPC applications using system-wide task scheduling[C/OL]//Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS). IEEE, 2024: 312-324. DOI: 10.1109/IPDPS57955.2024.00035.

[41] HUMPHRIES J T, NATU N, CHAUGULE A, et al. ghOSt: Fast & flexible user-space delegation of linux scheduling[C/OL]//Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP). ACM, 2021: 588-604. DOI: 10.1145/3477132.3483542.

[42] FRIED J, RUAN Z, OUSTERHOUT A, et al. Caladan: Mitigating interference at microsecond timescales[C/OL]//Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2020: 281-297. https://www.usenix.org/conference/osdi20/presentation/fried.

[43] QIN H, LI Q, SPEISER J, et al. Arachne: Core-aware thread management[C/OL]//Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2018: 145-160. https://www.usenix.org/conference/osdi18/presentation/qin.

[44] MILLER S, KUMAR A, VAKHARIA T, et al. Enoki: High velocity Linux kernel scheduler development[C/OL]//Proceedings of the European Conference on Computer Systems (EuroSys). ACM, 2024: 962-980. DOI: 10.1145/3627703.3629569.

[45] JIA Y, TIAN K, YOU Y, et al. Skyloft: A general high-efficient scheduling framework in user space[C/OL]//Proceedings of the ACM Symposium on Operating Systems Principles (SOSP). ACM, 2024: 265-279. DOI: 10.1145/3694715.3695973.

[46] LI Y, LAZAREV N, KOUFATY D, et al. LibPreemptible: Enabling fast, adaptive, and hardware-assisted user-space scheduling[C/OL]//Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA). IEEE, 2024: 922-936. DOI: 10.1109/HPCA57654.2024.00075.

[47] CHEN Z, ZHANG W, TANG Y, et al. Xkernel: Principled performance tunability of operating system kernels[C/OL]//Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2026: 547-565. https://www.usenix.org/conference/osdi26/presentation/chen-zhongjie.